The Georgette 150th - Image & Fulfilment Workflow

Operator guide: cataloguing photos and fulfilling print orders

Audience: operators registering prints and fulfilling Pixel Perfect orders. Application: exhibition.margies.app / local admin on port 3007.

1. Purpose

This guide describes how a photography exhibition print moves from a master TIFF on disk to a product on the public website, through Stripe checkout, automated print-file preparation, and finally submission to Pixel Perfect for production and shipping.

There are two distinct images in the system and they must not be confused:

- Master TIFF - the archival source used only for print production. Stored on the server under MASTER_FILES_DIR. Never served to the public website.

- Web JPEG - a smaller sRGB derivative shown on /shop and product pages. Stored under public/images/ and referenced from product_images.

Print size, paper, DPI, and lab metadata live on product_variants. Those fields are usually copied from variant_templates when the photo is registered.

2. Prerequisites before registering a photo

Before opening the admin Register Photo screen, confirm the following:

1. The finished master exists as a .tif or .tiff file in MASTER_FILES_DIR (production example: /mnt/exhibition-masters). In local development, MASTER_FILES_DIR_DEV may be used instead.

2. The master filename is path-free (for example isaac-rock-01.tif). The database stores the filename only - not a full folder path.

3. The TIFF includes an embedded ICC colour profile. The fulfilment worker refuses to process masters that have no embedded profile, because it will not guess a source colour space.

4. Variant templates (print sizes and base prices) already exist and are marked active. You will select which templates apply to this photo at registration time.

5. Stripe is configured in the environment, because registration creates Stripe products and prices for each selected size.

6. Optional but recommended: a readable title and a URL slug you are happy to keep (slugs must be unique).

3. Adding a photo to the website catalog (recommended path)

Use Admin -> Register Photo (/admin/register-photo). This is the primary path for exhibition prints.

Step-by-step:

1. Open /admin/register-photo while signed into the admin area.

2. Choose the master TIFF from the scanned list of files in MASTER_FILES_DIR. The UI shows file size and, where available, pixel dimensions and a suggested title/slug.

3. Confirm or edit the title, slug, description, location tag, photo type (Still camera / Drone / Underwater), featured flag, and edition size.

4. Select one or more print templates (variant sizes). You can leave template base prices or override a per-template price before submitting.

5. Web image - either leave blank so the system generates a JPEG from the master, or upload your own JPEG/PNG/WebP (max 8 MB). Generated webs are sRGB, EXIF-aware, max edge 2400 px, quality ~90.

6. Submit. The app then:

- Writes the web image to public/images/{slug}-{uuid}.jpg and creates a media_files record.
- Inserts a products row (product_type = print, is_available = true).
- Inserts product_variants by copying the selected variant_templates (width/height mm, border, paper, print DPI, finish, framing fields, and pricing) and stores the same master_filename on every variant.
- Inserts a primary product_images row pointing at the web JPEG path.
- Creates a Stripe product and price for each variant and saves stripe_price_id.

After a successful submit, the photo appears on /shop and is orderable at /shop/{slug}.

4. Alternative ways to register a product

PhotoLab / API registration - POST /api/products/register with a Bearer API key. Uses the same registerPrintProduct() logic, but the caller must supply an already-hosted web_image_url. No automatic web generation on that route.

Manual product editor - Admin -> Products -> New or Edit. Create the product and variants by hand. For each print variant, select a print template to pre-fill size and lab fields if desired. Upload shop images via the admin image upload endpoint. You must still set master_filename and positive width/height/print_dpi on print variants or fulfilment will fail later.

Use the Register Photo path whenever possible; it keeps master, web, variants, and Stripe in sync with one action.

5. What customers see and how ordering works

Shop listing (/shop) shows products where is_available is true. Product detail (/shop/{slug}) shows the primary web image and a variant selector (sizes and prices).

When the customer clicks Checkout:

1. The browser posts the selected variant_id (and quantity) to /api/checkout.

2. The app creates a Stripe Checkout Session and redirects the customer to Stripe.

3. After successful payment, Stripe sends checkout.session.completed to the app webhook.

4. The webhook creates an orders row (status paid) and order_items rows with fulfilment_status = awaiting_file.

5. Edition numbers are assigned for limited editions where applicable.

6. An order confirmation email is sent via Resend.

Important: no print JPEG is created at checkout time. Payment only queues the work. A separate Python worker prepares the lab file.

Admin can also create manual paid-style items; those also start as awaiting_file.

6. Automated print preparation (fulfilment worker)

The fulfilment worker (worker/fulfilment_worker.py) is intentionally separate from the Next.js web process. It should run on the server that can read MASTER_FILES_DIR (often under systemd or pm2).

Typical worker environment variables:

- EXHIBITION_API_BASE_URL - base URL of the exhibition app
- EXHIBITION_API_KEY - Bearer key for fulfilment API routes

- MASTER_FILES_DIR - same master folder the registrar used

- PRINT_OUTPUT_PROFILE_PATH - ICC file for lab output colour space (normally Adobe RGB 1998)

- GOOGLE_APPLICATION_CREDENTIALS and GOOGLE_DRIVE_FOLDER_ID - for Drive delivery

- Optional LOCAL_OUTPUT_DIR - write print files locally instead of uploading to Drive

- Optional WORKER_POLL_SECONDS - poll interval (default 60)

Each poll cycle:

1. GET /api/fulfilment/queue and find items with fulfilment_status = awaiting_file.

2. Locate MASTER_FILES_DIR / master_filename for that line item.

3. Convert colour from the TIFF's embedded profile to Adobe RGB 1998 (perceptual intent with black-point compensation).

4. Size the image to the variant's width_mm x height_mm at print_dpi, preserving aspect ratio (fit and centre on a white canvas). Add a white border if border_mm > 0.

5. Save a high-quality JPEG with the output ICC embedded.

6. Upload to a per-order Google Drive folder (or copy to LOCAL_OUTPUT_DIR).

7. PATCH the order item to fulfilment_status = file_ready, storing cloud_file_url and cloud_folder_path.

How size is known: the worker reads width/height/DPI from the ordered product_variants row (copied from templates at registration or set in the product editor). It does not look up variant_templates again.

7. Manual lab order and shipping

Use Admin -> Fulfilment (/admin/fulfilment).

Status meanings:

- awaiting_file - paid; waiting for the worker

- file_ready - print JPEG is available (open cloud_file_url)

- submitted_to_lab - admin has placed the Pixel Perfect order and saved the lab reference

- shipped - tracking number recorded; customer can be notified

- delivered - complete

Recommended admin steps after file_ready:

1. Open the Drive (or local) file and confirm size/paper/finish against the order line.

2. Place the order with Pixel Perfect using the prepared file and variant metadata (paper, finish, framing notes).

3. Save the Pixel Perfect order reference in the dashboard -> status becomes submitted_to_lab.

4. When the package leaves the lab (or you), enter tracking -> shipped, then notify the customer if desired.

Lifecycle timestamps file_ready_at, submitted_to_lab_at, and shipped_at are set when status changes.

8. Asset map (quick reference)

Master TIFF - print source - MASTER_FILES_DIR on disk - referenced by product_variants.master_filename

Web JPEG - shop display - public/images/... - referenced by product_images.image_url

Variant templates - reusable sizes/prices - exhibition.variant_templates

Product variants - sellable sizes for one photo - exhibition.product_variants (includes master_filename and

print geometry)

Order items - paid line items - exhibition.order_items (fulfilment_status drives the pipeline)

Print worker output - lab JPEG - Google Drive or LOCAL_OUTPUT_DIR - cloud_file_url on the order item

9. Common failure points

Master not found - file missing from MASTER_FILES_DIR, or filename in the database does not match the on-disk name.

Missing ICC in TIFF - worker stops with a clear error; re-export the master with an embedded profile.

No print dimensions on variant - width_mm / height_mm / print_dpi empty or zero; fix via product editor or re-register from templates.

Stripe misconfigured - registration fails creating prices; checkout will also fail if stripe_price_id is missing.

Worker not running - orders stay in awaiting_file indefinitely; check the Python process and API key.

Wrong Supabase / offline DB - admin and shop cannot load products; see the separate Supabase infrastructure guide.

10. Related repository docs

docs/image-and-fulfilment-workflow.md - markdown source of this workflow

docs/supabase-infrastructure.md - shared LAN Supabase (cashbook stack) notes

worker/README.md - how to install and run the fulfilment worker